Software Requirements Specification

for

Telegram Backed Hybrid Cloud Storage Management System

Version: 1.0

Status: Approved / Submission Draft

Prepared by: Swapnaraj Mohanty

SIC No.: 24BCSH93

Lab Roll No.: 11

Date: 03 September 2026

Name: Swapnaraj Mohanty
SIC No: 24BCSH93
Lab Roll No: 11
Date of Experiment: 3rd September 2026

Revision History

Name	Date	Reason for Changes	Version
Swapnaraj Mohanty	03 September 2026	Initial project-specific SRS prepared from the IEEE-modified template, project brief, and user stories.	1.0

Table of Contents

1. Introduction 4

1.1 Purpose 4

1.2 Document Conventions 4

1.3 Intended Audience and Reading Suggestions 4

1.4 Product Scope 4

1.5 References 5

2. Overall Description 5

2.1 Product Perspective 5

2.2 Product Functions 5

2.3 User Classes and Characteristics 6

2.4 Operating Environment 6

2.5 Design and Implementation Constraints 6

2.6 User Documentation 6

2.7 Assumptions and Dependencies 7

3. External Interface Requirements 7

3.1 User Interfaces 7

3.2 Hardware Interfaces 7

3.3 Software Interfaces 7

3.4 Communications Interfaces 8

4. System Features 8

4.1 User Registration and Authentication 8

Name: Swapnaraj Mohanty
SIC No: 24BCSH93
Lab Roll No: 11
Date of Experiment: 3rd September 2026

4.2 Dashboard and Virtual Folder Management....9
4.3 File Upload and Streaming....9
4.4 Telegram API Integration and Metadata Indexing....10
4.5 File Download and Deletion....11
4.6 Search and Workspace Usability....12
4.7 Administrative Metadata Export....12
5. Other Nonfunctional Requirements....13
5.1 Performance Requirements....13
5.2 Safety Requirements....13
5.3 Security Requirements....13
5.4 Software Quality Attributes....13
5.5 Business Rules....14
6. Other Requirements....14
6.1 Database Requirements....14
6.2 Data Retention and Local Persistence....14
6.3 Internationalization and Accessibility....14
Appendix A: Glossary....15
Appendix B: Analysis Models....15
B.1 Context Model....15
B.2 Core Data Relationship....15
B.3 Upload Data Flow....15
B.4 Download Data Flow....15
B.5 User-Story Dependency Summary....16
Appendix C: To Be Determined List....16

Note: Update the table of contents in Microsoft Word after opening the document.

Name: Swapnaraj Mohanty
SIC No: 24BCSH93
Lab Roll No: 11
Date of Experiment: 3rd September 2026

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional, interface, performance, security, quality, and other requirements for the Telegram Backed Hybrid Cloud Storage Management System, Version 1.0. The system is a middleware/proxy-based cloud storage application that provides a web-managed virtual file system while using a private Telegram channel as the remote file-storage backend and a local SQL database as the metadata layer.

The scope covers user authentication, virtual folder management, streamed file upload and download, Telegram file identifier capture, metadata indexing, file and folder operations, global file search, transfer progress reporting, error handling, and selected administrative capabilities.

The project brief identifies the central problem as the cost of commercial cloud storage and the lack of structured multi-user management in Telegram Bot API file hosting. The proposed system addresses this by mapping remote Telegram file references into user-specific virtual folder trees through an SQL metadata matrix.

1.2 Document Conventions

- The terms SHALL, MUST, SHOULD, and MAY are used to express requirement strength. SHALL/MUST indicates a mandatory requirement; SHOULD indicates a preferred requirement; MAY indicates an optional capability.
- Each functional requirement has a unique identifier beginning with FR.
- Each nonfunctional requirement has a unique identifier beginning with NFR.
- Priority values follow the project backlog terminology: Must Do, Should Do, and Could Do.
- TBD is used only where the supplied project materials do not define a required value or decision.

1.3 Intended Audience and Reading Suggestions

This document is intended for the project developer, project evaluator, testers, reviewers, and users who need a precise description of the system behavior. Developers should read Sections 2 through 4 first, followed by Section 5 for quality and security constraints. Testers should focus on Section 4 and the requirement identifiers. Reviewers may begin with Sections 1 and 2 and then inspect the detailed requirements.

1.4 Product Scope

The system provides a web dashboard through which authenticated users can maintain a private, virtualized file hierarchy. Files are transferred through a backend proxy to a private Telegram channel rather than being persistently stored on the application server. The SQL database stores the metadata and relationships needed to identify each remote object and present it in the user's folder tree.

- Account creation and authenticated access.
- User-specific storage isolation.
- Root folders and nested virtual subfolders.
- Dashboard navigation and parent-folder navigation.
- Asynchronous streamed file uploads.
- Telegram Bot API integration for remote file transport.

Name: Swapnaraj Mohanty
SIC No: 24BCSH93
Lab Roll No: 11
Date of Experiment: 3rd September 2026

- Persistent Telegram file_id capture and SQL metadata indexing.
- Streamed downloads from Telegram to the user's browser.
- File and empty-folder deletion.
- Global search across the user's dashboard.
- Upload progress and explicit transfer error feedback.
- Optional dark/light mode and SQL metadata export as backlog capabilities.

1.5 References

Reference	Description
BHAGWAT_SRS TEMPLATE-IEEE-Modified.doc	IEEE Software Requirements Specification template supplied for the project. It defines the section organization used by this document.
Telegram Backed Hybrid Cloud Storage Management System.pdf	Project brief containing the problem statement, functional dependencies, scope, and major constraints.
User Story.pdf	Project user-story backlog containing user stories, story points, priorities, risks, and dependency relationships.

2. Overall Description

2.1 Product Perspective

The product is a new, self-contained middleware application that combines a web interface, a Python-based application/backend layer, a relational SQL metadata store, and the Telegram Bot API. The system does not treat Telegram's raw file organization as the user's directory structure. Instead, the SQL metadata matrix provides the logical mapping Users → Folders → Files, while Telegram provides the remote file object.

High-level logical architecture:

- Web client/UI → authentication, dashboard, folder navigation, file actions, search, and progress/error display.
- Application/backend proxy → authentication/session handling, streaming, validation, Telegram API interaction, and business rules.
- SQL metadata database → users, folder hierarchy, file metadata, and relationships.
- Telegram Bot API/private channel → remote binary file transport and persistent Telegram file identifiers.

2.2 Product Functions

- Register a user with a unique username and password.
- Authenticate users and issue a JWT-based stateless session token.
- Prevent unauthenticated access to dashboard information.
- Render the current user's root folder and file contents.
- Create, enter, and nest virtual folders.
- Navigate back to parent directories.
- Select files through the upload interface and initiate asynchronous transfer.
- Stream upload data in chunks to avoid loading large files completely into server RAM.
- Use streaming fallback paths for files exceeding the standard Telegram Bot API upload ceiling.

Name: Swapnaraj Mohanty
SIC No: 24BCSH93
Lab Roll No: 11
Date of Experiment: 3rd September 2026

- Handle Telegram/API HTTP 429 throttling without crashing the application.
- Capture the Telegram file_id after successful transfer.
- Index file_id, file name, file size, and folder relationship in SQL metadata.
- Download files by looking up metadata and streaming the remote data to the browser.
- Delete file metadata records and unlink files from the virtual folder tree.
- Delete empty virtual folders.
- Search globally for files within the user's dashboard.
- Display upload progress and explicit network failure messages.
- Support optional light/dark appearance and administrative SQL metadata export.

2.3 User Classes and Characteristics

User Class	Characteristics	Main Capabilities
System User / Storage User	Authenticated end user managing personal cloud content.	Account, folders, uploads, downloads, deletion, search, navigation, progress and error feedback.
Unauthenticated Visitor	User who has not established a valid authenticated session.	Login/account-access screens only; dashboard content is not exposed.
System Administrator	Privileged operator for administrative functions defined by the project backlog.	Administrative SQL metadata export/backups. Exact administrative role model is TBD.

2.4 Operating Environment

The supplied materials specify a web interface, a Python application/runtime, an SQL metadata database, and the Telegram Bot API. Exact operating-system versions, browser support matrix, SQL implementation/version, deployment topology, and minimum hardware specifications are not defined in the source material and are therefore TBD.

Component	Specified Environment / Requirement
Client	Web browser capable of interacting with the web dashboard.
Application runtime	Python-based application/backend as indicated by the user stories.
Database	Relational SQL metadata database.
Remote storage/API	Telegram Bot API and private Telegram channel.
Network	Internet connectivity is required for Telegram-backed file transfers.
OS / Hardware minimums	TBD

2.5 Design and Implementation Constraints

- Telegram Bot API transfer limits and external API behavior constrain upload/download handling.
- The project brief states a standard HTTP upload limit of 50 MB and a standard download limit of 20 MB, with a 2 GB absolute cap via streaming/chunking.
- HTTP 429 throttling can occur and must be handled gracefully.
- Large uploads must be streamed rather than loaded completely into RAM.
- The intended backend flow avoids temporary or permanent local file persistence for transferred binary data.
- The SQL metadata matrix is essential to reconstruct the user's virtual storage hierarchy; loss of the database removes the mapping needed to identify remote files.
- User-specific isolation must be preserved across all file and folder queries.

Name: Swapnaraj Mohanty
SIC No: 24BCSH93
Lab Roll No: 11
Date of Experiment: 3rd September 2026

2.6 User Documentation

- End-user instructions for registration, login, folder creation/navigation, uploads, downloads, deletion, and search.
- Operational notes describing transfer limits, streaming behavior, and common error messages.
- Administrator instructions for metadata export if the optional administrative capability is implemented.
- Detailed deployment and environment documentation: TBD.

2.7 Assumptions and Dependencies

- A valid Telegram Bot API integration and access to the configured private channel are available.
- Users have network connectivity during remote file transfers.
- The SQL metadata database remains available and consistent with the application's schema.
- JWT session handling is available to support stateless authenticated sessions.
- The Telegram API may throttle requests; the application therefore depends on retry/cool-down handling.
- Remote Telegram files remain accessible through the stored identifiers/API mechanisms required by the application.

3. External Interface Requirements

3.1 User Interfaces

- Login/registration interface for account creation and authentication.
- Dashboard interface displaying folders and files belonging to the authenticated user.
- Empty-state dashboard placeholder when no files or folders exist.
- Folder row interaction that changes the displayed directory.
- Breadcrumb or 'Back to Parent' navigation control.
- Drag-and-drop/local file selection upload component.
- Upload progress indicator showing active percentage status.
- Download action associated with each listed file.
- File/folder deletion actions.
- Global file search input.
- Explicit error popup/message for upload/network failures.
- Optional global light/dark appearance toggle.

Exact visual dimensions, typography, accessibility conformance level, and final screen designs are not specified in the supplied sources and are TBD.

3.2 Hardware Interfaces

No specialized hardware interface is specified. The system is expected to run on ordinary client and server computing infrastructure capable of running the selected web/Python/SQL stack. Minimum hardware requirements are TBD.

3.3 Software Interfaces

Interface	Purpose	Data / Interaction
JWT authentication layer	Maintain stateless authenticated sessions.	Credentials in; JWT/session token out; authenticated requests carry token.
SQL database	Store user, folder, file metadata and	Users, parent-folder identifiers, file_id,

	relationships.	file name, file size and related metadata.
Telegram Bot API	Remote binary upload/download transport.	Streamed binary chunks, API responses, Telegram file_id, throttling/error responses.
Web browser	Present dashboard and initiate user actions.	HTTP requests/responses, upload/download streams, UI status and errors.

3.4 Communications Interfaces

The application communicates with the web client and Telegram Bot API using HTTP-based communication. The project explicitly depends on handling HTTP 429 responses from the Telegram API. Data transfer for large files is performed as a stream/chunk sequence rather than as a complete in-memory object.

- Authenticated web requests SHALL be associated with the authenticated user's identity.
- Telegram API failures SHALL be surfaced as controlled application outcomes rather than unhandled crashes.
- HTTP 429 responses SHALL cause controlled throttling/cool-down behavior.
- Exact timeout values, retry counts, TLS configuration details, and message schemas are TBD unless fixed by the implementation environment.

4. System Features

4.1 User Registration and Authentication

Description and Priority: High

Stimulus/Response Sequences:

1. User selects registration.
2. System requests username and password.
3. User submits credentials.
4. System validates uniqueness and creates the account.
5. User logs in and receives a JWT session token.

Functional Requirements:

FR-001: The system SHALL allow a visitor to create an account using a unique username and password.

Priority: Must Do

FR-002: The system SHALL reject account creation when the requested username is already associated with an account.

Priority: Must Do

FR-003: The system SHALL authenticate registered users before granting access to private dashboard data.

Priority: Must Do

FR-004: A successful login SHALL result in a JWT-based stateless session token.

Priority: Must Do

FR-005: The system SHALL prevent unauthenticated visitors from accessing dashboard views and SHALL redirect them to the login page.

Priority: Must Do

4.2 Dashboard and Virtual Folder Management

Description and Priority: High

Stimulus/Response Sequences:

6. Authenticated user opens dashboard.
7. System queries the current parent folder.
8. System renders matching folders/files.
9. User enters a folder or navigates back to its parent.
10. System refreshes the displayed contents.

Functional Requirements:

FR-006: The system SHALL display the authenticated user's current folder contents.

Priority: Must Do

FR-007: When the authenticated user has no files or folders, the system SHALL display an empty dashboard placeholder.

Priority: Should Do

FR-008: The system SHALL allow an authenticated user to create a custom-named virtual folder at the root directory level.

Priority: Must Do

FR-009: The system SHALL allow an authenticated user to select a folder and render only the folders/files belonging to that directory.

Priority: Must Do

FR-010: The system SHALL support nested virtual subfolders inside existing folders.

Priority: Must Do

FR-011: The system SHALL provide a 'Back to Parent' navigation action for returning to a higher folder level.

Priority: Should Do

FR-012: The system SHALL allow deletion of empty virtual folders.

Priority: Should Do

4.3 File Upload and Streaming

Description and Priority: Critical

Stimulus/Response Sequences:

11. User selects a local file.
12. Client initiates asynchronous upload.
13. Backend reads the file as chunks.
14. Backend sends chunks through the Telegram API.
15. System updates progress and handles throttling/errors.
16. On completion, the returned file_id is captured for indexing.

Functional Requirements:

FR-013: The system SHALL allow a storage user to select a local file through the upload interface and initiate an asynchronous upload into the current virtual folder.

Priority: Must Do

FR-014: The backend SHALL process large files as streamed/multipart chunks rather than loading the complete file into system RAM.

Priority: Must Do

FR-015: The upload pipeline SHALL stream incoming binary blocks directly toward the external Telegram Bot API endpoint without creating a persistent local copy of the file.

Priority: Must Do

FR-016: The application SHALL enforce the project's standard Telegram transfer ceiling of 50 MB for non-chunked uploads and SHALL use the appropriate streaming fallback for larger transfers.

Priority: Should Do

FR-017: The application SHALL support the project's stated maximum file size of 2 GB through streaming/chunking, subject to Telegram/API constraints.

Priority: Must Do

FR-018: The system SHALL display active upload progress as a percentage while an upload is being processed.

Priority: Should Do

FR-019: If network connectivity is lost during an active upload, the system SHALL present an explicit error message indicating that the transfer failed.

Priority: Should Do

4.4 Telegram API Integration and Metadata Indexing

Description and Priority: Critical

Stimulus/Response Sequences:

17. Telegram API receives the final chunk.
18. API returns transfer feedback and file identifier.
19. Backend captures file_id.
20. Metadata indexing writes the remote reference and file metadata into SQL.
21. Dashboard refresh makes the file visible.

Functional Requirements:

FR-020: After successful transmission of the final upload chunk, the system SHALL capture the persistent Telegram file_id returned by the Telegram API.

Priority: Must Do

FR-021: After successful file transmission, the system SHALL store the Telegram file_id, file name, file size, and relevant folder/user relationships in the SQL metadata database.

Priority: Must Do

FR-022: The system SHALL handle HTTP 429 throttling responses from the Telegram API by pausing for the designated cool-down duration rather than terminating the application.

Priority: Must Do

FR-023: The system SHALL treat successful metadata indexing as the condition that makes the uploaded remote file visible through the dashboard.

Priority: Must Do

4.5 File Download and Deletion

Description and Priority: Critical

Stimulus/Response Sequences:

22. User clicks Download.
23. System retrieves the file metadata.
24. Backend uses file_id to obtain remote content.
25. Binary data is streamed to the browser.
26. For deletion, the SQL metadata row is removed and the item disappears from the folder view.

Functional Requirements:

FR-024: The system SHALL provide a download action for each listed file record.

Priority: Must Do

FR-025: When a download is requested, the system SHALL look up the selected file's relational metadata and obtain the corresponding Telegram file_id.

Priority: Must Do

FR-026: The backend SHALL retrieve the remote file data from Telegram and stream the binary response directly to the user's browser.

Priority: Must Do

FR-027: The system SHALL allow an authenticated user to delete a file record from the SQL metadata database.

Priority: Must Do

FR-028: After deletion of a file record, the file SHALL no longer appear in the user's virtual folder tree.

Priority: Must Do

4.6 Search and Workspace Usability

Description and Priority: Medium

Stimulus/Response Sequences:

27. User enters search text.
28. System queries metadata across the user's accessible hierarchy.
29. Matching file records are returned and displayed.
30. Optional theme toggle changes the interface stylesheet.

Functional Requirements:

FR-029: The system SHALL provide a global file search input for locating files across the authenticated user's dashboard hierarchy.

Priority: Should Do

FR-030: Search results SHALL be restricted to metadata belonging to the authenticated user.

Priority: Should Do

FR-031: The system SHALL preserve the folder hierarchy when rendering search results or navigating to a selected file.

Priority: Should Do

FR-032: The system MAY provide a global light/dark mode stylesheet toggle.

Priority: Could Do

4.7 Administrative Metadata Export

Description and Priority: Low

Stimulus/Response Sequences:

31. Authorized administrator selects export.
32. System verifies administrative permission.
33. System exports the SQL metadata mapping.
34. System returns the backup to the administrator.

Functional Requirements:

FR-033: The system SHOULD provide an administrative export action that downloads a raw backup of the SQL metadata schema/mapping matrix.

Priority: Should Do

FR-034: The administrative export capability SHALL be restricted to authorized administrative users.

Priority: Should Do

FR-035: The exact administrator role-management mechanism is TBD.

Priority: TBD

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- NFR-001: Large file uploads SHALL be processed as a stream/chunk sequence so that the complete file does not need to reside in server RAM.
- NFR-002: The application SHOULD begin forwarding upload data without waiting for the complete file to be loaded on the server.
- NFR-003: Download responses SHOULD be streamed to the browser rather than requiring a complete local server-side copy.
- NFR-004: The application SHALL tolerate Telegram HTTP 429 throttling through controlled cool-down behavior.
- NFR-005: Exact throughput targets, maximum concurrent transfers, response-time targets, and retry intervals are TBD because the supplied sources do not specify numeric values.

5.2 Safety Requirements

- NFR-006: The application SHALL avoid persistent local storage of transferred file binaries where the specified streaming architecture permits.
- NFR-007: The application SHALL fail gracefully on transfer/network/API errors rather than terminating the entire service.
- NFR-008: The application SHALL prevent unauthorized users from accessing another user's virtual folder metadata.

5.3 Security Requirements

- NFR-009: Usernames SHALL uniquely identify user accounts within the system.
- NFR-010: Authentication SHALL use a JWT-based stateless session mechanism after successful login.

- NFR-011: Dashboard, file, and folder operations SHALL require a valid authenticated session.
- NFR-012: Database queries for user-owned resources SHALL enforce the authenticated user's ownership/isolation context.
- NFR-013: Telegram file identifiers SHALL be treated as protected backend metadata and SHALL not be exposed as an unrestricted replacement for application-level authorization.
- NFR-014: The administrative metadata export capability SHALL be authorization-protected.
- NFR-015: Password storage, password policy, JWT signing algorithm/secret management, transport-layer configuration, and exact encryption requirements are TBD because they are not specified in the supplied project documents.

5.4 Software Quality Attributes

Attribute	Requirement
Reliability	The application should continue operating when individual uploads encounter throttling or network failures.
Maintainability	The upload, Telegram integration, metadata indexing, authentication, and UI responsibilities should remain separable enough to permit independent maintenance.
Scalability	Streaming should reduce memory pressure from large files; exact concurrency/scaling limits are TBD.
Usability	Folder navigation, upload progress, download actions, and explicit errors should be visible and understandable.
Interoperability	The system must communicate with the Telegram Bot API and a relational SQL database.
Testability	Requirements are uniquely identified so that each behavior can be verified independently.
Portability	Specific supported operating systems and deployment environments are TBD.

5.5 Business Rules

- BR-001: Each user account must have a unique username.
- BR-002: A user's files and folders must be logically isolated from those of other users.
- BR-003: A folder belongs to a parent folder or the root level, forming a virtual hierarchy.
- BR-004: A file record is visible in the dashboard only when its required metadata has been indexed successfully.
- BR-005: File transfer behavior must respect the Telegram Bot API limits and throttling behavior stated by the project.
- BR-006: Deleting the local metadata record unlinks the remote Telegram file from the application's virtual tree; the supplied project brief explicitly warns that loss of the complete metadata database makes remote files unidentifiable through the application.

6. Other Requirements

6.1 Database Requirements

The system SHALL use a relational SQL metadata database implementing the logical relationships described by the project: Users → Folders → Files. At minimum, the metadata layer must support unique user identity, folder parent-child relationships, file-to-folder association, Telegram file_id, file name, and file size.

Name: Swapnaraj Mohanty
SIC No: 24BCSH93
Lab Roll No: 11
Date of Experiment: 3rd September 2026

Logical Entity	Required Information
User	Unique user identity, username, and authentication-related data.
Folder	Folder identity, folder name, owning user, and parent-folder relationship.
File	File identity, owning user/folder relationship, Telegram file_id, file name, and file size.

Exact SQL vendor, schema names, indexes, cascade rules, and backup format are TBD.

6.2 Data Retention and Local Persistence

The intended transfer path does not persist file binaries on local hard drives. The SQL database stores metadata required to map application-visible files to Telegram-hosted objects. The project brief identifies the metadata database as a critical dependency because deleting it breaks the mapping to remote files.

6.3 Internationalization and Accessibility

The supplied project documents do not define language localization, accessibility standards, or compliance targets. These requirements are TBD.

Name: Swapnaraj Mohanty
SIC No: 24BCSH93
Lab Roll No: 11
Date of Experiment: 3rd September 2026

Appendix A: Glossary

Term	Definition
API	Application Programming Interface.
Bot API	Telegram's programmatic interface used by the application for remote file transfer.
Dashboard	The authenticated web workspace where folders and files are displayed and managed.
File ID / file_id	Persistent identifier returned by Telegram for a successfully transmitted file and stored in application metadata.
JWT	JSON Web Token used by the system for stateless authenticated sessions.
Metadata	Information describing and locating a remote file in the application, including file_id, name, size, user and folder relationships.
Parent_Folder_ID	Logical reference used to identify the parent directory when rendering a folder's contents.
Private Telegram Channel	The Telegram-backed remote location used by the application for file hosting/transport.
Proxy	The backend layer that mediates communication between the web client and Telegram.
SQL	Structured Query Language used for the relational metadata database.
Streaming	Processing/transferring file data incrementally in chunks instead of loading the entire file into memory.
Virtual Folder	An application-level folder represented in SQL metadata rather than as a native Telegram directory.

Appendix B: Analysis Models

The following logical models are derived from the supplied project scope and user-story dependencies. They are included as textual models because the source documents do not provide finalized UML/DFD/ER diagrams.

B.1 Context Model

User / Browser ↔ Web Application ↔ Python Backend Proxy ↔ Telegram Bot API / Private Channel; the Python backend also ↔ SQL Metadata Database.

B.2 Core Data Relationship

Users 1 —— * Folders 1 —— * Files, with Folders recursively related through Parent_Folder_ID.

B.3 Upload Data Flow

35. User selects file in web UI.
36. Backend receives file as streamed multipart chunks.
37. Backend forwards chunks to Telegram Bot API.
38. Telegram returns transfer feedback and final file_id.
39. Backend writes file metadata to SQL.
40. Dashboard refreshes and displays the indexed file.

B.4 Download Data Flow

41. User selects a file and requests download.

Name: Swapnaraj Mohanty
SIC No: 24BCSH93
Lab Roll No: 11
Date of Experiment: 3rd September 2026

42. Backend retrieves metadata and file_id from SQL.
43. Backend requests remote data from Telegram.
44. Backend streams the binary response to the browser.

B.5 User-Story Dependency Summary

User Story ID	Depends On
7	12
20	7
1	7
3	7
13	3
14	3, 13
2	7, 13
8	21
9	8, 21
24	8
18	8
16	18
10	8, 21
4	16
11	4, 16
25	16
23	2, 16, 18
5	13, 14
9	3, 14
22	16, 14
17	2, 8
6	16

Appendix C: To Be Determined List

TBD ID	Open Item
TBD-001	Supported operating-system versions and minimum server/client hardware.
TBD-002	Supported browser versions and exact UI accessibility requirements.
TBD-003	SQL database vendor/version and finalized schema names/indexing strategy.
TBD-004	Exact Telegram API retry counts, retry intervals, and timeout values.
TBD-005	Exact performance targets for upload/download throughput and concurrent transfers.
TBD-006	Password policy and password hashing/storage specification.
TBD-007	JWT signing algorithm, expiration period, refresh/revocation strategy, and secret/key management.
TBD-008	TLS/HTTPS deployment and certificate requirements.
TBD-009	Administrator role-management and authorization mechanism.
TBD-010	Detailed user-interface screen specifications and accessibility/localization requirements.
TBD-011	Backup/restore procedure beyond the optional SQL metadata export user story.

Name: Swapnaraj Mohanty
SIC No: 24BCSH93
Lab Roll No: 11
Date of Experiment: 3rd September 2026